

To check Docker images, visit the official repository located at <https://hub.docker.com>. It hosts many images and provides detailed information about them, like their description, supported tags, Docker files and much more.

Listing images: To list all downloaded images, use the following command:

```
# docker images
```

Pull image: As the name suggests, this command downloads the image from a remote repository and stores it on the local disk. To download the image, we have to provide the image's name as an argument. For instance, the commands given below pull the *busybox* image:

```
# docker pull busybox
Using default tag: latest ----->
latest: Pulling from library/busybox
d070b8ef96f6: Pull complete
Digest: sha256:2107a35b58593c58ec5f4e8f2c4a70d195321078aeb
fadfbfb223a2ff4a4ed21
Status: Downloaded newer image for busybox:latest
```

Using tags: If we don't provide any additional option, then the *pull* command downloads an image tagged with the *latest* tag. We can see it in the previous command's output, where it has printed *Using default tag: latest*. To pull an image with a specific tag, we can provide the tag name with the *pull* command. For instance, the command given below pulls an image with the tag *1.28.1-uclibc*:

```
# docker pull busybox:1.28.1-uclibc
1.28.1-uclibc: Pulling from library/busybox ----->
----->
Digest: sha256:2c3a381fd538dd732f20d824f07fac1e300a9ef56eb
4006816fa0cd992e85ce5
Status: Downloaded newer image for busybox:1.28.1-uclibc
```

We can get image tags from the Docker Hub located at <https://hub.docker.com>.

Getting the history of an image: Using the *history* command, we can retrieve historical data about the image like its ID, creation date, author, size, description and so on. For instance, the following command shows the history of the 'busybox' image:

```
# docker history busybox
```

Deleting an image: Like containers, we can also delete Docker images. Docker provides the 'rm' command for this purpose. In this command, 'i' stands for image. For instance, to delete a 'busybox' image, execute the following command:

```
# docker rmi f6e427c148a7
```

 **Note:** We have to provide an image ID to it, which we can obtain using the *docker images* command.

Advanced Docker topics

So far we have explored only the basics of Docker. This can be a good start for beginners. However, the discussion does not end here. Docker is a feature-rich application. So let's now briefly discuss some advanced Docker concepts.

Docker Compose: Docker Compose can be used to deploy and configure an entire software stack using automated methods rather than using the *docker run* command, followed by manual configuration. We can define the configuration in a YAML file and use that to perform deployment. Shown below is a simple example of configuration:

```
version: '2.0'

services:
  database:
    image: "jarvis/acme-web-app"
  web:
    image: "mysql"
```

In the above 'docker-compose.yml' file, we have defined the configuration under the 'services' dictionary. We have also provided the images that should be used for deployment.

To deploy the above configuration, execute the following command in a terminal:

```
# docker-compose up
```

To stop and destroy a deployed configuration, execute the following commands in the terminal:

```
# docker-compose stop
# docker-compose down
```

Mapping ports: Like any other application, we can run Web applications inside a container. But the challenge is how to allow access to outside users. For this purpose, we can provide port mapping using the '-p' option, as follows:

```
# docker run -p 80:5000 jarvis/acme-web-app
```

In the above example, Port 80 from the host machine is mapped to Port 5000 for the 'acme-web-app' container. Now, users can access this Web application using the host machine's IP address.

Continued on page...62